



CMPT 295

Unit - Machine-Level Programming

Lecture 10 – Assembly language basics: **move** operation

Demo and SOLUTION to Homework

Demo – Swap Function

```
void swap(long *xp, long *yp)
{
    long L1 = *xp;
    long L2 = *yp;
    *xp = L2;
    *yp = L1;
    return;
}
```

```
swap:
    movq    (%rdi), %rax    # L1 = *xp
    movq    (%rsi), %rdx    # L2 = *yp
    movq    %rdx, (%rdi)    # *xp = L2
    movq    %rax, (%rsi)    # *yp = L1
    ret
```

Registers

%rdi	
%rsi	
%rax	
%rdx	

Memory => Remember:

Address

0x0020	
0x0018	
0x0010	
0x0008	
0x0000	

*Compressed
view of memory*

Demo – Swap Function

Parameter-to-register mapping:
xp -> %rdi, yp -> %rsi

```
void swap(long *xp, long *yp)
{
    long L1 = *xp;
    long L2 = *yp;
    *xp = L2;
    *yp = L1;
    return;
}
```

```
swap:
    movq    (%rdi), %rax    # L1 = *xp
    movq    (%rsi), %rdx    # L2 = *yp
    movq    %rdx, (%rdi)    # *xp = L2
    movq    %rax, (%rsi)    # *yp = L1
    ret
```

Registers

%rdi	0x0020
%rsi	0x0000
%rax	123
%rdx	456

Memory

Address	Memory
0x0020	123 456
0x0018	
0x0010	
0x0008	
0x0000	456 123

1. Result of `movq (%rdi), %rax`

3 2. Result of `movq (%rsi), %rdx`

3. Result of
`movq %rdx, (%rdi)`

4. Result of
`movq %rax, (%rsi)`

Homework

- ▶ Since we cannot do memory-memory transfer with a single `mov*` instruction ...
 - ▶ Can you write a little **x86-64** assembly program that transfers data stored at address `0x0000` to address `0x0018` ?

Registers

%rdi	
%rsi	
%rax	
%rdx	

Memory

Address	
0x0020	
0x0018	
0x0010	
0x0008	
0x0000	6

Homework - Solution

- Since we cannot do memory-memory transfer with a single `mov*` instruction ...
- Can you write a little **x86-64** assembly program that transfers data stored at address `0x0000` to address `0x0018` ?

Registers

%rdi	0x0000
%rsi	0x0018
%rax	6
%rdx	

Memory

Address	
0x0020	
0x0018	6
0x0010	
0x0008	
0x0000	6

```
transfer:
    movq    $0x0000,%rdi    # get 1st address into %rdi
    movq    $0x0018,%rsi    # get 2nd address into %rsi
    movq    (%rdi), %rax    # load content of 1st address into %rax
    movq    %rax, (%rsi)    # store %rax into 2nd address
    ret
```